



PROGRAMMATION PAR CONTRAINTES — EPITA SCIA 2026

Composition melodique sous contraintes

Générer des mélodies uniquement avec des règles.

Sujet H1 · Samuel Krief et Nicolas Teisseire

La musique est faite de regles

La musique tonale obeit a des regles precises

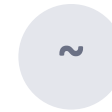
Héritant de siècles de théorie : rester dans une gamme, eviter certains intervalles, finir sur la tonique...

C'est exactement un problème de contraintes

On decrit les regles, le solveur CP-SAT trouve une solution qui les respecte toutes.

La difference clé

Chaque mélodie produite respecte les règles *par construction* — aucune n'est filtrée après coup.



Modele génératif

Apprend un style sur un corpus. Aucune garantie : peut produire des fausses notes.



Notre approche : CP

Encode les règles explicitement. Garantit le respect des regles musicales.

Le probleme vu comme un CSP

On génère une suite de 16 entiers. Ces entiers sont des hauteurs de notes — mais pour le solveur, ce sont juste des nombres.

X

Variables

Une variable par note : pitch[0] ... pitch[15]

D

Domaines

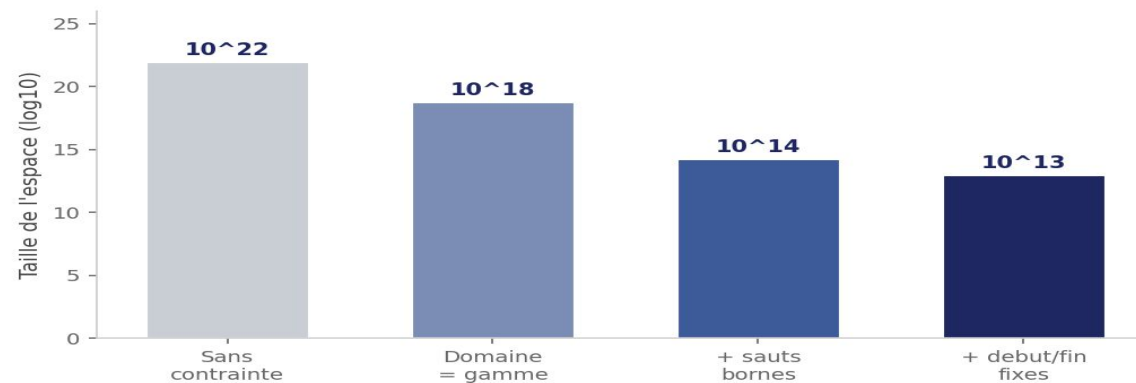
Notes de la gamme dans une tessiture : ~15 valeurs
au lieu de 128

C

Contraintes

Les regles musicales : dures (obligatoires) et souples (preferences)

Principe CP : plus on réduit les domaines en amont, plus le solveur est rapide. On restreint donc chaque variable a la gamme des le depart.



5 règles essentielles

Une contrainte dure doit être satisfaite, sinon il n'y a pas de solution.

- 1** **1re et dernière note = tonique** *Ancre la mélodie dans sa tonalité*
- 2** **Avant-dernière = dominante ou sensible** *Force une cadence finale qui conclut*
- 3** **Sauts bornés à 7 demi-tons** *Une quinte maximum : reste chantable*
- 4** **Pas deux notes identiques de suite** *Évite les répétitions triviales*
- 5** **Interdiction du triton (6 demi-tons)** *Le 'diabolus in musica' — trop dissonant*

Zoom : le triton

Au Moyen-Age, ce saut de 6 demi-tons était interdit dans la musique liturgique.

On le surnommait diabolus in musica — le diable en musique.

C'est typiquement une règle 'métier' qu'un CSP gère naturellement : $\text{abs}(\text{saut}) \neq 6$.

Des preferences, pas des obligations

Une soft constraint est une preference : on paie un coût si elle est violée.
Le solveur cherche à minimiser la somme totale des coûts.

$$\min \sum w_i \times \text{violation}_i$$

C'est le Weighted CSP du notebook CSP-7. On l'a decliné en 6 preferences.

4 preferences melodiques

- Favoriser les petits intervalles
- Assurer un ambitus suffisant
- Encourager les changements de direction
- Eviter les oscillations A-B-A-B

2 preferences structurelles

- Consonance sur les temps forts (pesanteur métrique)
- Contour melodique en arche (sommet aux 2/3)

Modéliser des concepts musicaux abstraits

strong_beat

Pesanteur métrique

Sur les temps forts (1er et 3e temps), on préfère les notes stables de la triade tonique :
Do-Mi-Sol.

COTE CP-SAT

AddAllowedAssignments + OnlyEnforceIf

arch

Contour en arche

La mélodie monte vers un sommet placé aux 2/3 de la phrase, puis redescend vers la cadence.

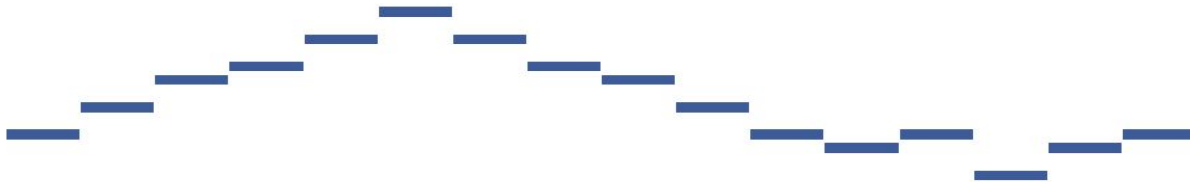
COTE CP-SAT

variable max + booléens is_peak[t]

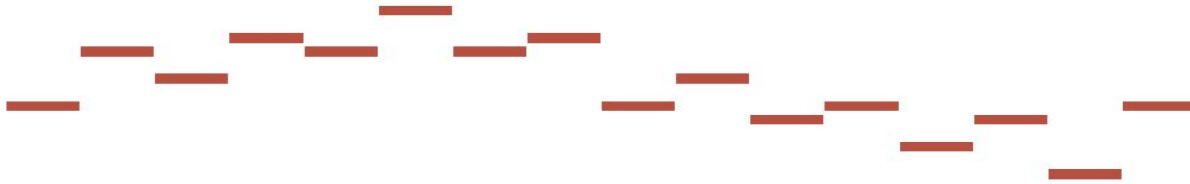
Trois profils, trois styles

Un profil = un jeu de poids. En changeant uniquement les coefficients, on obtient des styles distincts sans toucher au modèle.

Profil fluide



Profil aventureux



Profil minimaliste



Fluide

Poids fort sur smoothness, consonance, arche. Style classique, presque comptine.



Aventureux

Favorise les sauts et un grand ambitus. Contour libre.



Minimaliste

Tout à zero sauf smoothness. Sobre, sans direction marquée.

C'est notre démonstration scientifique principale.

Le mode variation

Le probleme inverse : au lieu de générer une mélodie depuis zéro, on fournit un theme partiel et le solveur complete les trous.



Theme partiel (notes fixees + trous)

Solveur CP-SAT
complete en respectant toutes les contraintes

Pourquoi c'est intéressant

- **Probleme avec assignments partielles**
 - On ajoute juste des contraintes d'égalité : `model.Add(pitch[t] == valeur)`
- **C'est le mécanisme d'un Sudoku partiellement rempli**
 - Quelques cases donnees, le solveur déduit le reste
- **Génère des variations sur un thème**
 - Comme le faisaient Bach ou Mozart dans leurs variations

Ecouter et lire les mélodies



MIDI

Fichier audio : on ECOUTE directement la mélodie dans n'importe quel lecteur.



Notation ABC

Format texte rendu en partition graphique sur abcjs.net : on LIT la melodie.

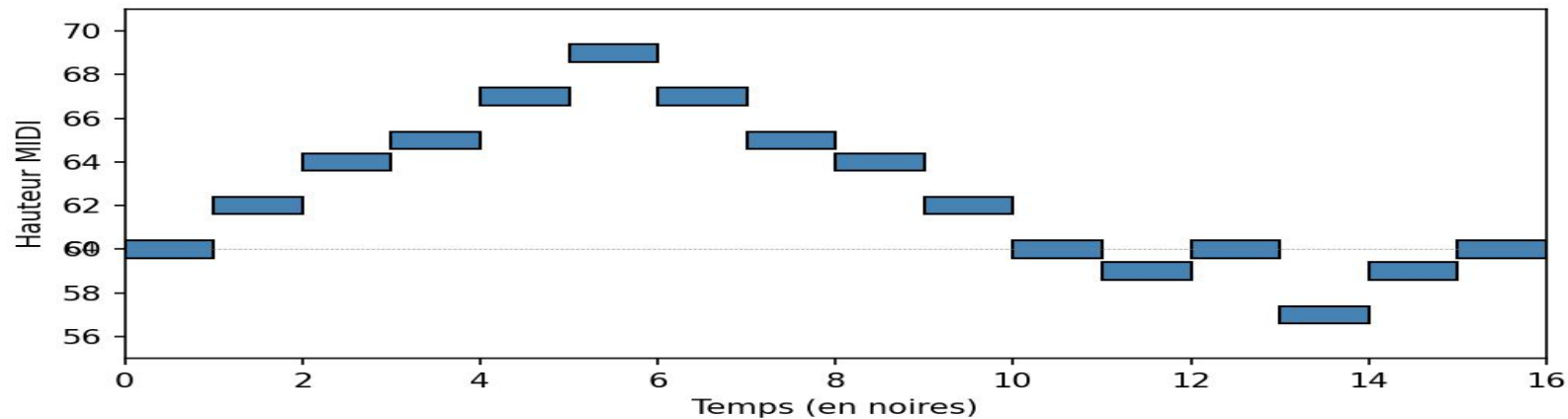
Verification objective

7 / 7 contraintes dures OK

88% de consonance sur temps forts

pic mélodique dans la zone cible

Exemple généré avec le profil fluide :



DEMO LIVE

Generer une melodie, la coller sur abcjs.net, écouter le MIDI.

Une architecture simple et lisible

music_theory.py

Le savoir musical : gammes, conversions MIDI. Zero logique CP.

solver.py

Le coeur CP-SAT : variables, contraintes dures et souples, profils.

export.py

Les sorties : MIDI, notation ABC, piano-roll matplotlib.

En chiffres

~500

lignes de Python

6 + 6

contraintes dures + souples

3

gammes : Do maj, Sol maj, La min

1

solveur : OR-Tools CP-SAT

Notions du cours mobilisées : formalisation X-D-C (CSP-1), optimisation et variables auxiliaires (CSP-5), Weighted CSP (CSP-7).

Limitations

Des choix de périmètre clairs — et justifiables.



Pas de polyphonie

On traite une seule voix. Une 2e voix ouvrirait sur le contrepoint et ses contraintes de consonance.



Rythme fixe

Toutes les notes durent une noire. La pesanteur métrique est simulée via `strong_beat`.



Pas d'évaluation humaine

Plutôt que juger le 'beau', on vérifie objectivement le respect des règles.

Ces limitations sont des perspectives d'extension naturelles. Le modèle est conçu pour les accueillir sans être cassé.

CONCLUSION

Ce que ce projet démontre

- ✓ La composition musicale se modélise naturellement en CSP : variables, domaines, contraintes.
- ✓ Les contraintes dures garantissent la validité ; les souples paramètrent le style.
- ✓ En changeant juste les poids, on obtient des styles distincts — sans changer le modèle.
- ✓ Le mode variation illustre un problème inverse, comme un Sudoku partiellement rempli.

L'essentiel : la programmation par contraintes permet d'encoder une connaissance experte avec des garanties — là où un modèle boîte-noire n'en donne aucune.

Perspectives d'extension

01

Polyphonie

Ajouter une 2e voix (basse) avec contraintes de consonance — on entre dans le contrepoint.

02

Rythme variable

Variables duration[t] et contraintes metriques : somme des durées = nombre de mesures.

03

Ornementation

Insertion automatique de notes de passage entre les pivots du mode variation.

Le modèle actuel est conçu pour accueillir chacune de ces dimensions de façon modulaire.

MERCI DE VOTRE ATTENTION

Questions ?

Composition mélodique sous contraintes · Sujet H1 · EPITA SCIA 2026

